

Java Backend Interview Series

Sub-Chapter 1.12: `java.util.concurrent`

ExecutorService | Locks | Atomic | Concurrent Collections | CompletableFuture

52 Q&A | Advanced Questions | Exercises | Cheat Sheet

1. ExecutorService & Thread Pool Internals

The **Executor framework** decouples task submission from execution. **ExecutorService** extends **Executor** with lifecycle management (shutdown, awaitTermination) and the ability to submit **Callable** tasks returning **Future** results. **ThreadPoolExecutor** is the workhorse — all **Executors** factory methods delegate to it.

```
// ThreadPoolExecutor – full constructor
ThreadPoolExecutor pool = new ThreadPoolExecutor(
4, // corePoolSize
16, // maximumPoolSize
60L, TimeUnit.SECONDS, // keepAliveTime for idle threads above core
new LinkedBlockingQueue<>(1000), // workQueue (bounded!)
new ThreadFactory() { // custom thread factory
AtomicInteger n = new AtomicInteger();
public Thread newThread(Runnable r) {
Thread t = new Thread(r, "worker-" + n.incrementAndGet());
t.setDaemon(false);
return t;
}
},
new ThreadPoolExecutor.CallerRunsPolicy() // rejection policy
);

// Executors factory shortcuts
ExecutorService fixed = Executors.newFixedThreadPool(4);
ExecutorService cached = Executors.newCachedThreadPool(); // unbounded!
ExecutorService single = Executors.newSingleThreadExecutor();
ScheduledExecutorService sched = Executors.newScheduledThreadPool(2);
ExecutorService vt = Executors.newVirtualThreadPerTaskExecutor(); // Java 21

// Lifecycle
pool.shutdown(); // stop accepting, finish queued tasks
pool.shutdownNow(); // interrupt running tasks, drain queue
boolean done = pool.awaitTermination(30, TimeUnit.SECONDS);

// submit vs execute
pool.execute(runnable); // fire-and-forget
Future<String> f = pool.submit(callable); // returns Future
List<Future<R>> futures = pool.invokeAll(callables); // all tasks
R first = pool.invokeAny(callables); // first successful result
```

Factory Method	Core	Max	Queue	Use Case
<code>newFixedThreadPool(n)</code>	n	n	Unbounded LinkedBQ	CPU-bound, known concurrency
<code>newCachedThreadPool()</code>	0	MAX_INT	SynchronousQueue	Short-lived bursts (risk OOM)
<code>newSingleThreadExecutor()</code>	1	1	Unbounded LinkedBQ	Sequential execution
<code>newScheduledThreadPool(n)</code>	n	MAX_INT	DelayedWorkQueue	Scheduled/periodic tasks
<code>newVirtualThreadPerTask()</code>	N/A	N/A (JVM)	None	High-concurrency I/O (Java 21)

ThreadPoolExecutor Lifecycle: Task Submission Flow

1. If running threads < corePoolSize: create a new thread.
2. Else if queue not full: enqueue the task.
3. Else if running threads < maxPoolSize: create a new (non-core) thread.
4. Else: apply RejectedExecutionHandler (Abort, CallerRuns, Discard, DiscardOldest).

2. Future, CompletableFuture & Async Pipelines

Future<V> represents a result that will be available later. Its `get()` blocks until complete. **CompletableFuture<V>** (Java 8) adds non-blocking callbacks, composition, and exception handling — enabling full async pipelines without blocking.

```
// CompletableFuture basics
CompletableFuture<String> cf = CompletableFuture
    .supplyAsync(() -> fetchUser(id), executor) // async supply
    .thenApply(user -> user.toUpperCase()) // transform result
    .thenApply(s -> "Hello, " + s) // chain transform
    .exceptionally(ex -> "Error: " + ex.getMessage()) // handle exception
    .thenAccept(System.out::println); // consume result

// Combining futures
CompletableFuture<String> user = supplyAsync(() -> getUser(id));
CompletableFuture<String> account = supplyAsync(() -> getAccount(id));
CompletableFuture<String> both = user
    .thenCombine(account, (u, a) -> u + " | " + a); // wait for both
CompletableFuture<Object> either = user
    .applyToEither(account, String::toUpperCase); // whichever finishes first

// allOf / anyOf
CompletableFuture.allOf(f1, f2, f3)
    .thenRun(() -> System.out.println("All done"));

// Exception handling
CompletableFuture<String> safe = cf
    .handle((result, ex) -> ex != null ? "default" : result) // always called
    .whenComplete((r, ex) -> log(r, ex)); // side-effect

// Timeout (Java 9+)
cf.orTimeout(5, TimeUnit.SECONDS)
    .completeOnTimeout("fallback", 5, TimeUnit.SECONDS);

// Async variants (run callback on a different thread)
cf.thenApplyAsync(fn) // uses ForkJoinPool.commonPool()
    .thenApplyAsync(fn, executor); // uses custom executor
```

Method	Input -> Output	Notes
<code>thenApply(fn)</code>	<code>T -> U</code>	Sync transform; same thread
<code>thenApplyAsync(fn)</code>	<code>T -> U</code>	Async transform; pool/executor
<code>thenCompose(fn)</code>	<code>T -> CF[U]</code>	Flat-map; avoids nested CFs
<code>thenCombine(cf,fn)</code>	<code>T,U -> V</code>	Combine two independent CFs
<code>exceptionally(fn)</code>	<code>Throwable->T</code>	Recover from exception
<code>handle(fn)</code>	<code>(T,ex)->U</code>	Always called; handles both
<code>whenComplete(fn)</code>	<code>(T,ex)->void</code>	Side-effect; propagates result

3. ReentrantLock, ReadWriteLock & StampedLock

ReentrantLock provides all synchronized semantics plus: timed/interruptible lock attempts, fairness, and Condition objects (multiple wait queues per lock). **ReentrantReadWriteLock** allows concurrent reads but exclusive writes. **StampedLock** (Java 8) adds optimistic reads for high-read workloads.

```
// ReentrantLock – explicit lock with try/finally
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
```

```
// critical section
} finally {
    lock.unlock(); // ALWAYS in finally
}

// tryLock with timeout -- avoids deadlock
if (lock.tryLock(100, TimeUnit.MILLISECONDS)) {
    try { /* work */ } finally { lock.unlock(); }
} else { /* handle lock unavailable */ }

// Condition variables -- multiple wait queues
ReentrantLock lock2 = new ReentrantLock();
Condition notFull = lock2.newCondition();
Condition notEmpty = lock2.newCondition();

// Producer
lock2.lock();
try {
    while (isFull()) notFull.await();
    add(item);
    notEmpty.signal(); // wake one consumer
} finally { lock2.unlock(); }

// ReentrantReadWriteLock
ReadWriteLock rwLock = new ReentrantReadWriteLock();
Lock readLock = rwLock.readLock(); // multiple concurrent readers
Lock writeLock = rwLock.writeLock(); // exclusive write

// StampedLock -- optimistic reads (no read-lock needed if no write in progress)
StampedLock sl = new StampedLock();
long stamp = sl.tryOptimisticRead(); // non-blocking
double x = this.x, y = this.y; // read fields
if (!sl.validate(stamp)) { // check if write occurred during read
    stamp = sl.readLock();
    try { x = this.x; y = this.y; }
    finally { sl.unlockRead(stamp); }
}
```

Lock Type	Concurrent Readers	Interruptible	Timed tryLock	Optimistic Read
synchronized	No	No	No	No
ReentrantLock	No	Yes	Yes	No
ReentrantReadWriteLock	Yes (read)	Yes	Yes	No
StampedLock	Yes (read)	Yes	Yes	Yes

4. Atomic Variables & Non-Blocking Algorithms

The **java.util.concurrent.atomic** package provides lock-free, thread-safe variables built on Compare-And-Swap (CAS). They are more efficient than synchronization for simple updates under low-to-moderate contention.

```
// AtomicInteger -- lock-free counter
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // atomic i++
counter.getAndAdd(5); // atomic i += 5
counter.compareAndSet(5, 10); // CAS: if value==5, set to 10
int snapshot = counter.get();

// AtomicReference -- lock-free object reference
AtomicReference<Config> configRef = new AtomicReference<>(new Config());
Config old, fresh;
```

```

do {
    old = configRef.get();
    fresh = old.withNewSetting("key", "val");
} while (!configRef.compareAndSet(old, fresh)); // spin until CAS wins
// AtomicStampedReference -- solves ABA problem
AtomicStampedReference<String> ref =
    new AtomicStampedReference<>("initial", 0);
int[] stampHolder = new int[1];
String value = ref.get(stampHolder); // gets value and current stamp
ref.compareAndSet(value, "updated", stampHolder[0], stampHolder[0] + 1);
// LongAdder -- high-throughput counter (better than AtomicLong under contention)
LongAdder adder = new LongAdder();
adder.increment(); // distributed cells, merged on sum()
long total = adder.sum(); // eventual consistency (not point-in-time exact)
// LongAccumulator -- generalised LongAdder with custom operator
LongAccumulator maxVal = new LongAccumulator(Long::max, Long.MIN_VALUE);
maxVal.accumulate(42);
long max = maxVal.get();
// VarHandle (Java 9+) -- low-level atomic on any field
VarHandle VH = MethodHandles.lookup()
    .findVarHandle(MyClass.class, "count", int.class);
VH.compareAndSet(obj, expected, newVal);
VH.getAndAdd(obj, delta);

```

LongAdder vs AtomicLong

AtomicLong uses a single CAS-updated cell. Under high contention, threads spin retrying CAS. LongAdder maintains a Cell array -- different threads update different cells, reducing contention. sum() merges all cells. Use LongAdder for high-throughput counters/accumulators. Use AtomicLong when you need a consistent point-in-time value or CAS semantics.

5. Concurrent Collections

The `java.util.concurrent` package provides high-performance thread-safe collections. They use fine-grained locking, CAS, or copy-on-write rather than coarse-grained synchronization, and do NOT throw `ConcurrentModificationException` during iteration.

```

// ConcurrentHashMap -- segmented/striped locking (Java 8: CAS + synchronized per bucket)
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
map.put("key", 1);
map.putIfAbsent("key", 2); // atomic check-then-act
map.computeIfAbsent("key", k -> k.length()); // atomic compute
map.merge("key", 1, Integer::sum); // atomic merge
map.compute("key", (k, v) -> v == null ? 1 : v + 1); // atomic compute
// Aggregation -- NOT atomic snapshot but useful for stats
map.forEach((k, v) -> process(k, v));
long count = map.mappingCount(); // approximate for large maps
map.search(1, (k, v) -> v > 100 ? k : null); // parallel search
// CopyOnWriteArrayList -- reads lock-free; writes copy the array
CopyOnWriteArrayList<String> cowList = new CopyOnWriteArrayList<>();
cowList.add("item"); // creates new underlying array
// Iteration always sees a consistent snapshot (even if list is modified)
for (String s : cowList) { process(s); } // no ConcurrentModificationException
// BlockingQueue implementations
BlockingQueue<Task> bounded = new ArrayBlockingQueue<>(1000); // bounded, array
BlockingQueue<Task> unbounded = new LinkedBlockingQueue<>(); // unbounded, linked
BlockingQueue<Task> priority = new PriorityBlockingQueue<>(); // priority-ordered

```

```

BlockingQueue<Task> sync = new SynchronousQueue<>(); // no internal buffer (handoff)
TransferQueue<Task> transfer = new LinkedTransferQueue<>(); // producer waits for consumer
// ConcurrentLinkedQueue -- lock-free unbounded FIFO
ConcurrentLinkedQueue<String> clq = new ConcurrentLinkedQueue<>();
clq.offer("item"); // never blocks
String head = clq.poll(); // null if empty
// ConcurrentSkipListMap/Set -- sorted, concurrent (like TreeMap but thread-safe)
ConcurrentSkipListMap<String, Integer> sorted = new ConcurrentSkipListMap<>();
sorted.headMap("M").entrySet(); // all keys < "M"
sorted.floorKey("N"); // largest key <= "N"

```

Collection	Ordering	Blocking	Null Keys/Values	Best For
ConcurrentHashMap	None	No	No nulls	High-concurrency map
CopyOnWriteArrayList	Index	No	Yes	Read-heavy, rare writes
ArrayBlockingQueue	FIFO	Yes	No nulls	Bounded producer-consumer
LinkedBlockingQueue	FIFO	Yes	No nulls	Unbounded or bounded queue
PriorityBlockingQueue	Priority	Yes	No nulls	Priority-based task queue
SynchronousQueue	Handoff	Yes	No nulls	Direct handoff (zero buffer)
ConcurrentLinkedQueue	FIFO	No	No nulls	Lock-free unbounded queue
ConcurrentSkipListMap	Sorted	No	No nulls	Sorted concurrent map

6. Synchronization Utilities: CountdownLatch, CyclicBarrier, Semaphore, Exchanger

The `java.util.concurrent` package provides higher-level coordination primitives for common concurrency patterns.

```

// CountdownLatch -- one-shot countdown; cannot be reset
CountDownLatch startLatch = new CountDownLatch(1);
CountDownLatch doneLatch = new CountDownLatch(5);
// Workers wait for a start signal then count down when done
for (int i = 0; i < 5; i++) {
    executor.submit(() -> {
        startLatch.await(); // wait for signal
        doWork();
        doneLatch.countDown(); // signal this worker is done
    });
}
startLatch.countDown(); // fire the starting gun
doneLatch.await(); // wait for all workers
// CyclicBarrier -- reusable; all threads wait until all arrive
CyclicBarrier barrier = new CyclicBarrier(4, () ->
    System.out.println("All threads reached barrier -- phase complete"));
// Each thread calls:
barrier.await(); // blocks until 4 threads have called await()
// Semaphore -- controls access to N permits (resource pool)
Semaphore sem = new Semaphore(10); // 10 concurrent connections allowed
sem.acquire(); // blocks if no permit available
try { useConnection(); }
finally { sem.release(); }
// Semaphore as binary mutex (permits=1)
Semaphore mutex = new Semaphore(1);

```

```
// Unlike ReentrantLock, can be released by a different thread!
// Exchanger -- two threads swap objects at a rendezvous point
Exchanger<DataBuffer> exchanger = new Exchanger<>();
// Thread 1 (producer):
DataBuffer full = fillBuffer();
DataBuffer empty = exchanger.exchange(full); // swap with consumer
// Thread 2 (consumer):
DataBuffer received = exchanger.exchange(new DataBuffer()); // get full, give empty
process(received);
// Phaser -- flexible CyclicBarrier+CountDownLatch replacement
Phaser phaser = new Phaser(3); // 3 parties
phaser.arriveAndAwaitAdvance(); // arrive and wait for others
phaser.arriveAndDeregister(); // leave the phaser
phaser.register(); // dynamically add a party
```

Utility	Reusable	Count Direction	Use Case
CountDownLatch	No	Down to 0	One-shot: wait for N events / start signal
CyclicBarrier	Yes	Resets at 0	Phased computation: all threads sync at checkpoint
Semaphore	Yes	Acquire/Release	Resource pool, rate limiting, binary mutex
Exchanger	Yes	N/A (2-thread)	Two-thread buffer swap / pipeline handoff
Phaser	Yes	Flexible	Dynamic party count, multi-phase tasks

7. ForkJoinPool, Work-Stealing & Parallel Streams

ForkJoinPool implements a **work-stealing** algorithm where idle threads steal tasks from the back of busy threads' dequeues. It is optimised for divide-and-conquer recursive tasks. Java's parallel streams and `CompletableFuture` async tasks run on `ForkJoinPool.commonPool()` by default.

```
// RecursiveTask -- returns a value
class SumTask extends RecursiveTask<Long> {
    private final long[] arr;
    private final int lo, hi;
    static final int THRESHOLD = 10_000;
    SumTask(long[] arr, int lo, int hi) {
        this.arr = arr; this.lo = lo; this.hi = hi;
    }
    @Override
    protected Long compute() {
        if (hi - lo <= THRESHOLD) {
            long sum = 0;
            for (int i = lo; i < hi; i++) sum += arr[i];
            return sum;
        }
        int mid = (lo + hi) / 2;
        SumTask left = new SumTask(arr, lo, mid);
        SumTask right = new SumTask(arr, mid, hi);
        left.fork(); // async submit left
        long rightResult = right.compute(); // compute right inline
        long leftResult = left.join(); // wait for left
        return leftResult + rightResult;
    }
}
```

```
ForkJoinPool pool = new ForkJoinPool(Runtime.getRuntime().availableProcessors());
long total = pool.invoke(new SumTask(array, 0, array.length));
```

```
// Custom ForkJoinPool for parallel streams (avoid commonPool for I/O)
ForkJoinPool customPool = new ForkJoinPool(8);
long result = customPool.submit(() ->
    LongStream.rangeClosed(1, 1_000_000)
        .parallel()
        .filter(n -> isPrime(n))
        .count()
    ).get();

// Work-stealing: idle thread T2 steals from the TAIL of busy T1's deque
// T1 pushes tasks at head (LIFO for cache locality)
// T2 steals from tail (FIFO -- larger chunks first)
```

ForkJoinPool.commonPool() Caution

parallel streams and `CompletableFuture.supplyAsync()` use `commonPool()` by default. I/O-bound tasks in `commonPool` block carrier threads and starve CPU-bound tasks. Always use a custom executor for I/O: `CompletableFuture.supplyAsync(task, ioExecutor)`. `commonPool` parallelism = `Runtime.availableProcessors() - 1` (configurable via `-Djava.util.concurrent.ForkJoinPool.common.parallelism=N`).

8. ScheduledExecutorService, Timeouts & Rate Limiting

ScheduledExecutorService supports delayed and periodic task execution. It replaces `Timer` (which has issues with exceptions killing the timer thread). Combined with `Semaphore` or `RateLimiter` patterns, it forms the basis for rate-limiting.

```
ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

// One-time delay
scheduler.schedule(() -> sendAlert(), 5, TimeUnit.SECONDS);

// Fixed-rate: next run at startTime + n*period (may overlap if slow)
scheduler.scheduleAtFixedRate(
    () -> collectMetrics(),
    0, // initial delay
    10, // period
    TimeUnit.SECONDS
);

// Fixed-delay: next run starts period after previous completes (no overlap)
scheduler.scheduleWithFixedDelay(
    () -> processQueue(),
    0, 5, TimeUnit.SECONDS
);

// Timeout pattern with CompletableFuture
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> slowOperation())
    .orTimeout(3, TimeUnit.SECONDS) // Java 9: throws TimeoutException
    .completeOnTimeout("default", 3, TimeUnit.SECONDS); // Java 9: completes with value

// Simple rate limiter using Semaphore (token-bucket style)
public class RateLimiter {
    private final Semaphore permits;
    private final ScheduledExecutorService scheduler;

    public RateLimiter(int permitsPerSecond) {
        this.permits = new Semaphore(0);
        this.scheduler = Executors.newSingleThreadScheduledExecutor();
        scheduler.scheduleAtFixedRate(
            () -> permits.release(permitsPerSecond),
            0, 1, TimeUnit.SECONDS
        );
    }
}
```

```
public void acquire() throws InterruptedException {  
    permits.acquire();  
}  
}
```

Interview Q&A; - Standard Questions

Q1. What is the Executor framework and why is it preferred over raw threads?

The Executor framework decouples task submission from execution mechanics. Benefits: (1) **Thread reuse** — pools avoid costly thread creation for every task; (2) **Lifecycle management** — shutdown/awaitTermination for clean shutdown; (3) **Future results** — submit(Callable) returns Future for async results; (4) **Bounded resources** — bounded queues prevent OOM under load; (5) **Monitoring** — getActiveCount(), getCompletedTaskCount(). Key types: Executor, ExecutorService, ScheduledExecutorService, ThreadPoolExecutor.

Q2. What are the four ThreadPoolExecutor rejection policies?

(1) **AbortPolicy** (default): throws RejectedExecutionException — caller must handle. (2)

CallerRunsPolicy: runs the task on the calling thread — natural back-pressure. (3) **DiscardPolicy**: silently drops the task — data loss risk. (4) **DiscardOldestPolicy**: drops the oldest queued task and retries submission. Custom: implement RejectedExecutionHandler — e.g., log + queue to a database, or apply a rate limiter. CallerRunsPolicy is safest for preventing data loss.

Q3. What is the difference between shutdown() and shutdownNow()?

shutdown(): initiates an orderly shutdown — no new tasks accepted, but all previously submitted tasks (including queued ones) complete. Non-blocking. **shutdownNow()**: attempts to halt running tasks via interrupt(), drains the queue (returns unstarted tasks as a List), and returns immediately. Running tasks may not stop if they ignore interruption. Best practice: call shutdown(), then awaitTermination(timeout), then shutdownNow() if not done.

Q4. What is CompletableFuture and how does it differ from Future?

Future<V> has get() which blocks until complete — no callbacks. It cannot be manually completed.

CompletableFuture<V> adds: non-blocking callbacks (thenApply/thenAccept), composition (thenCompose for flat-map), combination (thenCombine), exception handling (exceptionally/handle), manual completion (complete(v), completeExceptionally(ex)), timeout (orTimeout, Java 9+). It implements both Future and CompletionStage interfaces.

Q5. What is the difference between thenApply and thenCompose?

thenApply(fn): fn maps T to U (like Stream.map) — result is CompletableFuture<U>. If fn itself returns a CompletableFuture, you get CompletableFuture<CompletableFuture<U>>. **thenCompose(fn)**: fn maps T to CompletableFuture<U> and the result is flattened to CompletableFuture<U> (like Stream.flatMap). Use thenCompose when chaining async operations: getUser(id).thenCompose(user -> getOrders(user.id)).

Q6. How does ConcurrentHashMap differ from HashMap and Hashtable?

HashMap: not thread-safe. Hashtable: coarse-grained synchronization (every operation locks the whole map). ConcurrentHashMap: Java 8 uses CAS + per-bucket synchronized blocks (no global lock). Key differences: (1) null keys/values NOT allowed; (2) iterators are weakly consistent (show state at creation + any subsequent updates); (3) size() is approximate; (4) atomic compound operations: putIfAbsent, computeIfAbsent, merge; (5) much higher read throughput than Hashtable.

Q7. What is a BlockingQueue and when would you use it?

BlockingQueue is a thread-safe queue with blocking semantics: **put(e)** blocks when full; **take()** blocks when empty; offer(e, timeout) and poll(timeout) have timed versions. It is the canonical producer-consumer implementation — eliminates manual wait/notify. Implementations: ArrayBlockingQueue (bounded), LinkedBlockingQueue (optionally bounded), PriorityBlockingQueue (priority ordered), SynchronousQueue (direct handoff, zero capacity). Used by ThreadPoolExecutor as its work queue.

Q8. What is the difference between CountdownLatch and CyclicBarrier?

CountDownLatch: one-shot, non-resettable countdown. Threads await() until count reaches 0. Count can only decrease. Use case: wait for N events, or a start signal (latch of 1). CyclicBarrier: reusable barrier — N threads each call await(); all block until all N arrive; then all proceed together (optionally running a barrier action). Resets automatically. Use case: phased simulation, iterative algorithms where all threads sync between phases.

Q9. What is ReentrantLock and what advantages does it have over synchronized?

ReentrantLock implements Lock with additional features: (1) **Timed tryLock**: tryLock(timeout) avoids indefinite blocking; (2) **Interruptible**: lockInterruptibly() throws InterruptedException on interrupt; (3) **Fair mode**: ReentrantLock(true) for FIFO ordering; (4) **Multiple Conditions**: newCondition() for per-condition wait queues (vs single wait queue with synchronized); (5) **Diagnostics**: isLocked(), getQueueLength(), isHeldByCurrentThread(). Downside: must unlock in finally; more verbose than synchronized.

Q10. What is a Semaphore and what are its use cases?

A Semaphore maintains a set of N permits. acquire() blocks until a permit is available; release() returns a permit. Unlike ReentrantLock, a Semaphore can be released by a different thread than the one that acquired it. Use cases: (1) **Resource pool** — limit concurrent DB connections, HTTP connections; (2) **Rate limiting** — N requests per time window; (3) **Binary semaphore (N=1)** — mutex that can be released by another thread (e.g., signal pattern).

Q11. What is ForkJoinPool and how does work-stealing work?

ForkJoinPool is designed for recursive divide-and-conquer tasks (RecursiveTask/RecursiveAction). Each worker thread has a double-ended deque (deque). Workers push/pop from their own deque's head. **Work-stealing**: idle threads steal tasks from the tail of other threads' deques. This balances load without a central queue bottleneck. Head access is LIFO (cache-local), tail stealing is FIFO (larger subtasks stolen first). Used by: parallel streams, CompletableFuture.supplyAsync (default), RecursiveTask.

Q12. What is the difference between ArrayBlockingQueue and LinkedBlockingQueue?

ArrayBlockingQueue: fixed-size circular array; single lock for put and take (higher contention). Bounded only. Pre-allocates all memory upfront. Predictable memory. LinkedBlockingQueue: linked-list nodes; separate locks for put and take (higher throughput); optionally bounded (unbounded by default). Nodes are GC-allocated per element — GC overhead. For high-throughput producer-consumer: LinkedBlockingQueue (separate put/take locks). For predictable bounded memory: ArrayBlockingQueue.

Q13. What is CopyOnWriteArrayList and when should you use it?

CopyOnWriteArrayList: on every write (add, set, remove), a new copy of the underlying array is created. Reads are completely lock-free (iterate on the snapshot taken at iterator creation). Write-heavy workloads are expensive: O(n) per write + GC pressure. Use when: reads vastly outnumber writes (e.g., event listener registries, caches). Iterators never throw ConcurrentModificationException. Do NOT use for write-heavy collections — use ConcurrentLinkedQueue or synchronised lists.

Q14. What is StampedLock and how does optimistic reading work?

StampedLock (Java 8) returns a long stamp for each lock operation. tryOptimisticRead() returns a stamp without acquiring any lock. After reading fields, validate(stamp) checks whether any write occurred. If valid: no lock overhead for the read. If invalid: fall back to readLock(). Trade-offs: faster reads in low-contention scenarios; but not reentrant (unlike ReentrantLock); cannot upgrade from read to write lock without releasing read lock.

Q15. How does LongAdder achieve better throughput than AtomicLong?

AtomicLong uses a single volatile cell — all threads CAS-compete on it, causing cache line ping-pong. LongAdder maintains a base value plus a Cell[] array. Under contention, threads are hashed to different cells and update locally. sum() = base + sum(cells). This reduces CAS contention to near-zero by spreading updates. Trade-off: sum() is not a point-in-time snapshot — reads may not reflect concurrent increments. Best for high-throughput counters; not for exact CAS-based logic.

Q16. What is the difference between `invokeAll()` and `invokeAny()`?

`invokeAll(tasks)`: submits all tasks, blocks until all complete (or timeout), returns `List<Future<V>>` — all futures are done (either completed or cancelled). **`invokeAny(tasks)`**: submits all tasks, returns the result of the FIRST successful task, cancels remaining tasks. Throws `ExecutionException` if all fail, `TimeoutException` on timeout. Use `invokeAll` for gather patterns; `invokeAny` for racing alternatives (first cache wins).

Q17. What happens if you don't call `Future.get()` on a task that throws an exception?

If a submitted `Callable` throws an exception, it is captured and stored in the `Future`. The exception is NOT propagated until `get()` is called — `get()` wraps it in `ExecutionException`. If you never call `get()`, the exception is silently swallowed. This is a common bug: `pool.submit(callable)` without `get()` loses exceptions. Fix: always call `future.get()` or add exception logging via `CompletableFuture.handle()/exceptionally()`, or a custom `afterExecute` in `ThreadPoolExecutor`.

Q18. What is `SynchronousQueue` and when would you use it?

`SynchronousQueue` has zero capacity — each `put()` blocks until a `take()` is ready, and vice versa. It is a direct handoff mechanism — the producer hands data directly to a waiting consumer. Used by: `Executors.newCachedThreadPool()` (tasks are directly handed to idle threads or trigger new thread creation). Use case: when you want a tight coupling between producer and consumer (no buffering), or when tasks should only be processed when a thread is immediately ready.

Q19. How do you implement a timeout for a `CompletableFuture` in Java 9+?

Java 9 added two methods: (1) **`orTimeout(duration, unit)`**: if the CF does not complete within the duration, it completes exceptionally with `TimeoutException`. (2) **`completeOnTimeout(value, duration, unit)`**: if the CF does not complete within the duration, it completes normally with the given value (fallback). Both use a shared JVM delay scheduler (no extra thread needed). For Java 8: use `ScheduledExecutorService` to call `cf.completeExceptionally(new TimeoutException())`.

Q20. What is the `Phaser` class and how does it differ from `CyclicBarrier`?

`Phaser` is a more flexible version of `CyclicBarrier` + `CountDownLatch` combined. Differences: (1) **Dynamic registration** — parties can register/deregister at runtime; (2) **Multi-phase** — tracks phase number, automatically advances to next phase; (3) **Tiered hierarchies** — `Phasers` can have a parent for tree structures; (4) **Termination** — arrives at 0 parties, returns negative phase number. Use `Phaser` for complex multi-phase concurrent algorithms with varying participant counts.

Q21. What is a `ScheduledExecutorService` and how does it replace `Timer`?

`Timer` uses a single thread — if a task throws an exception, the `Timer` thread dies and no more tasks run. `ScheduledExecutorService` uses a thread pool — one failing task doesn't affect others. `Timer` tasks run serially — late tasks delay subsequent ones. `ScheduledExecutorService`: `scheduleAtFixedRate` (skips if slow) vs `scheduleWithFixedDelay` (waits for completion). Supports `Callable` (returns `Future`). Always prefer `ScheduledExecutorService` over `Timer` in production code.

Q22. What is the `Exchanger` class and give a use case?

`Exchanger` synchronises two threads at a rendezvous point where they swap objects. `exchange(V)` blocks until the other thread also calls `exchange()`, then returns the other's object. Use case: double-buffer pipeline — one thread fills a buffer while another processes the previous one. At each exchange point they swap: the filler gets the empty buffer, the processor gets the full one. Another use case: genetic algorithms where two threads swap chromosomes for crossover.

Q23. How does `ConcurrentHashMap.compute()` differ from `get()` then `put()`?

`get()-then-put()` is a compound operation — NOT atomic. Two threads can both `get()` the same value, both compute a new value, and one update is lost. `compute(key, remappingFunction)` is atomic — the function is applied under lock, guaranteeing no other thread modifies that key during the computation. Similarly: `computeIfAbsent` (only if key missing), `computeIfPresent` (only if key present), `merge` (merge with existing value). These eliminate the need for external synchronization on `ConcurrentHashMap` operations.

Q24. What is an ExecutionException and how do you handle it properly?

ExecutionException wraps a checked or unchecked exception thrown by a Callable submitted to an ExecutorService. future.get() throws ExecutionException — the actual cause is via getCause(). Best practice: try { result = future.get(); } catch (ExecutionException e) { Throwable cause = e.getCause(); if (cause instanceof MyException me) throw me; else throw new RuntimeException(cause); } Java 21 tip: future.resultNow() returns result without blocking (throws if not done/failed).

Q25. What is the default thread pool used by CompletableFuture.supplyAsync()?

By default, supplyAsync() uses ForkJoinPool.commonPool(). The commonPool parallelism is Runtime.getRuntime().availableProcessors() - 1 (minimum 1). Problem: commonPool is shared across the JVM — I/O-blocking tasks starve CPU tasks. Best practice: always pass a custom executor for I/O-bound tasks: CompletableFuture.supplyAsync(task, ioExecutor). With virtual threads (Java 21): Executors.newVirtualThreadPerTaskExecutor() is ideal for I/O chains.

Q26. What is the difference between fixed-rate and fixed-delay scheduling?

scheduleAtFixedRate(task, delay, period): next execution starts at initialTime + n*period — regardless of how long the task takes. If task takes longer than period, next run starts immediately after (no overlap by default — ScheduledThreadPool won't fire a second instance of the same task).

scheduleWithFixedDelay(task, delay, period): next execution starts period time AFTER the previous task COMPLETES. Guaranteed gap between runs. Use fixed-rate for regular heartbeats; fixed-delay for cleanup tasks that must not overlap.

Interview Q&A; - Advanced Questions

[ADV] Q27. Explain how ConcurrentHashMap is implemented in Java 8 (not segment-based).

Java 8 ConcurrentHashMap replaced Java 7's 16-segment approach with per-bucket synchronization. Internal: array of Node<K,V> bins. For reads: volatile array reads -- no lock needed (Node.val/next are volatile). For writes to an empty bin: CAS on the array slot (no lock). For writes to a non-empty bin: synchronized on the first node of that bin. At 8+ collisions: bins convert to TreeBin (red-black tree) for O(log n) lookups. Resize: concurrent transfer -- threads cooperate to move buckets to the new array (ForwardingNode sentinel).

[ADV] Q28. How does ForkJoinPool's work-stealing differ from a standard fixed thread pool?

Fixed thread pool: all threads share a single central queue (LinkedBlockingQueue). Under high task count, queue access becomes a bottleneck (lock contention). ForkJoinPool: each thread has its own double-ended deque. Push/pop from own deque head (LIFO). Steal from tail of another thread's deque (FIFO). Benefits: (1) no central bottleneck; (2) tasks stolen in FIFO order means larger subtasks go to idle threads; (3) worker's own LIFO access is cache-friendly (recently pushed task is hot). Ideal for recursive tasks; overkill for uniform independent tasks.

[ADV] Q29. What is the AQS (AbstractQueuedSynchronizer) architecture and how does ReentrantLock use it?

AQS maintains: (1) a volatile int state (semantics defined by subclass); (2) a CLH queue of Node objects representing blocked threads. ReentrantLock: state=0 means unlocked; state=N means locked with N reentrant acquisitions by one thread. tryAcquire: CAS state 0->1 (non-fair) or check queue (fair). If failed: enqueue in CLH, call LockSupport.park(). unlock: CAS state decrement; if 0, unpark head.next via LockSupport.unpark(). Condition.await(): releases lock, enqueues in condition queue, parks; signal() moves node from condition queue back to CLH.

[ADV] Q30. What causes thread pool saturation and how do you handle it gracefully?

Pool saturation occurs when: all threads are busy AND the work queue is full. Result:

RejectedExecutionHandler is invoked. Graceful handling strategies: (1) **CallerRunsPolicy**: back-pressure -- calling thread executes the task, slowing submission. (2) **Custom handler**: write to a persistent queue (DB, Kafka) for later retry. (3) **Circuit breaker**: detect saturation, return error to client quickly. (4) **Bounded queue + monitoring**: alert when queue depth exceeds threshold. (5) **Virtual threads**: for I/O-bound work, use virtual threads -- no saturation.

[ADV] Q31. How does CompletableFuture.thenApplyAsync vs thenApply affect the thread of execution?

thenApply(fn): runs fn on the thread that completed the previous stage (or the calling thread if already complete). Can accidentally run heavy computation on the I/O thread. thenApplyAsync(fn): always submits fn to the default pool (ForkJoinPool.commonPool()) or a specified executor. Ensures the callback runs on a separate thread. Pattern: use thenApplyAsync(fn, executor) for CPU-heavy transforms to avoid blocking the thread that completed the upstream stage.

[ADV] Q32. What is the difference between a conditional variable (Condition) and Object.wait/notify?

Object.wait/notify uses the object's single intrinsic condition queue -- notifyAll wakes ALL waiters even if only one condition is met, causing spurious work. Condition (from Lock.newCondition()): multiple condition queues per lock -- signal() wakes only threads waiting on that specific condition. Example: BoundedBuffer can have notFull.signal() in take() and notEmpty.signal() in put() -- consumers wake only producers and vice versa. More precise signalling reduces unnecessary wakeups and contention.

[ADV] Q33. How does CopyOnWriteArrayList's iterator work, and what are the consistency guarantees?

At iterator creation, CopyOnWriteArrayList snapshots the underlying array reference. The iterator always iterates over this snapshot -- it is immune to concurrent modifications. Iterator does NOT support remove(). Consistency: the iterator sees the state of the list at the moment of iterator creation. Subsequent adds/removes to the list are invisible to the iterator (no CME, but potentially stale). This is a 'snapshot isolation' model -- acceptable for event listener traversal, not suitable if you need real-time consistency.

[ADV] Q34. What is the LinkedTransferQueue and how does it differ from LinkedBlockingQueue?

LinkedTransferQueue implements TransferQueue which adds transfer(e) and tryTransfer(e). transfer(e): if a consumer is already waiting, hand off directly (like SynchronousQueue); otherwise enqueue and block until consumed. tryTransfer(e): handoff only if a consumer is waiting; returns false immediately otherwise. LinkedTransferQueue is unbounded and uses a lock-free algorithm (Michael-Scott queue with dual nodes). Better throughput than LinkedBlockingQueue in high-contention scenarios. Used in ForkJoinPool's task submission for virtual thread compatibility.

[ADV] Q35. How does the ForkJoinPool commonPool affect virtual threads in Java 21?

Virtual threads in Java 21 use a dedicated ForkJoinPool as their scheduler (separate from commonPool). The virtual thread scheduler's parallelism = available processors. CompletableFuture chains on virtual threads: if you use Executors.newVirtualThreadPerTaskExecutor() as the executor, each CF stage runs on a fresh virtual thread -- I/O blocking unmounts the carrier thread. Still avoid mixing CPU-bound work in virtual thread pools -- creates many carrier threads doing CPU work with work-stealing overhead.

[ADV] Q36. Explain the internals of PriorityBlockingQueue.

PriorityBlockingQueue is an unbounded blocking queue backed by a binary heap (min-heap). Elements must be Comparable or a Comparator must be provided. offer()/add(): O(log n) sift-up. poll()/take(): O(log n) sift-down from root. Concurrent access: single ReentrantLock for all operations (unlike LinkedBlockingQueue's 2 locks). No upper bound -- can grow indefinitely (potential OOM under high load). take() blocks on an empty queue using a Condition. drainTo() is NOT guaranteed to drain in priority order -- use with care.

[ADV] Q37. What is a CompletionService and when would you use it over invokeAll?

CompletionService (ExecutorCompletionService) wraps an ExecutorService with a BlockingQueue of completed futures -- futures are available in completion order (not submission order). invokeAll() blocks until ALL tasks complete. CompletionService: poll()/take() retrieves results as they complete -- lets you process results while others are still running. Use case: N price-check API calls -- process the first 3 that return, cancel the rest. cs.submit(task) then cs.take() in a loop processes results optimally.

[ADV] Q38. What memory ordering guarantees does ConcurrentHashMap provide?

ConcurrentHashMap provides **happens-before** guarantees: A put(k,v) happens-before any subsequent get(k) that returns v. This is because Node.val and Node.next are volatile, and array reads are volatile. get() performs a volatile read of the array slot -- observes the most recent put(). For compound operations: compute/merge/computeIfAbsent are atomic under the bin lock and provide HB between the function execution and subsequent reads. Cross-key atomicity is NOT guaranteed -- 'snapshot' of multiple keys is not atomic.

[ADV] Q39. How does ScheduledThreadPoolExecutor handle tasks that throw exceptions?

If a periodic task (scheduleAtFixedRate/scheduleWithFixedDelay) throws an unchecked exception, the task is silently cancelled -- no more executions, no logging unless you check the Future. This is a common production bug: a periodic health-check fails silently. Fix: (1) wrap the task body in try/catch and handle exceptions; (2) override afterExecute(Runnable, Throwable) in a custom ThreadPoolExecutor subclass; (3) call future.get() periodically to detect failures. One-time delayed tasks (schedule()) also behave the same way.

[ADV] Q40. What is the difference between a lock-free and obstruction-free algorithm?

Lock-free: system-wide progress guaranteed -- at least one thread completes in finite steps even if others are delayed. CAS retry loops: if one thread's CAS fails, another must have succeeded. **Obstruction-free**: a weaker guarantee -- a thread makes progress only if it runs alone (no interference). Practical in low-contention scenarios; a contention manager backs off conflicting threads. **Wait-free**: strongest -- every thread completes in bounded steps. Java's AtomicInteger, ConcurrentLinkedQueue are lock-free. True wait-free algorithms exist (Kogan-Petrunk queue) but are complex.

[ADV] Q41. How does ThreadPoolExecutor's prestartAllCoreThreads() affect performance?

By default, core threads are created lazily (only when a task arrives). prestartAllCoreThreads() starts all corePoolSize threads immediately on pool creation. prestartCoreThread() starts one. Benefits: eliminates thread-creation latency on first burst of tasks -- important for latency-sensitive services that receive bursts at startup. Downside: idle threads consume memory even during warm-up. Also consider: allowCoreThreadTimeOut(true) -- lets core threads expire too (saves memory during idle periods).

[ADV] Q42. What happens internally when a ConcurrentHashMap resizes?

ConcurrentHashMap uses cooperative resizing. When any thread triggers resize, it creates a new larger table (2x). A ForwardingNode sentinel is placed in each bucket after it is transferred. Multiple threads cooperatively transfer buckets: each thread picks up a strideSize chunk (default 16) of buckets to transfer. Reads: if get() encounters a ForwardingNode, it follows to the new table. Writes: if put() encounters ForwardingNode, it helps transfer before inserting. This cooperative design means resize does not block operations -- fully concurrent.

[ADV] Q43. How can you implement a non-blocking stack using AtomicReference?

A lock-free Treiber Stack using CAS: push: create new node, CAS top from current to new node (retry on failure). pop: read top, CAS top from top to top.next (retry on failure). This is ABA-safe for push/pop because popped nodes are not reused (Java GC reclaims them). Performance: O(1) per op, lock-free -- practical for high-concurrency stacks. Drawback: under extreme contention, CAS retries can cause livelock-like degradation. Use AtomicStampedReference if node reuse creates ABA risk.

[ADV] Q44. What is the significance of the 'transfer' method in LinkedTransferQueue?

transfer(e) implements a synchronous handoff if a consumer is already waiting, otherwise it enqueues and blocks until consumed. This bridges the gap between SynchronousQueue (zero-capacity, always blocks until consumer ready) and LinkedBlockingQueue (always enqueues). The dual-node algorithm: unfulfilled requests/data slots are stored as nodes with a 'is-data' marker. A producer's node matches a waiting consumer's node via CAS. Used in ForkJoinPool as an efficient work queue.

[ADV] Q45. How does invokeAny() cancel remaining tasks after first success?

invokeAny() submits all tasks to the executor, uses an ExecutorCompletionService internally to wait for results. On the first successful result, it cancels all other futures via future.cancel(true) -- sending interrupt signals. Tasks must be written to respond to interruption (check isInterrupted(), handle InterruptedException). If all tasks fail (all throw exceptions), invokeAny throws ExecutionException with the last exception. A version with timeout: invokeAny(tasks, timeout, unit) throws TimeoutException if no task succeeds.

[ADV] Q46. What is a Disruptor pattern and how does it outperform BlockingQueue?

The LMAX Disruptor is a high-performance inter-thread messaging framework using a lock-free ring buffer. Key techniques: (1) Ring buffer pre-allocated with fixed slots (no GC allocation); (2) Sequence numbers (not pointers) for head/tail -- no CAS on linked nodes; (3) Single writer principle per producer slot -- eliminates CAS under single-producer mode; (4) Cache-line padding on sequences to eliminate false sharing; (5) Busy-spin or yield strategy instead of blocking. Throughput: ~25M ops/sec vs ~5M for LinkedBlockingQueue on typical hardware.

[ADV] Q47. Explain the memory visibility semantics of volatile in the context of ConcurrentHashMap's get().

ConcurrentHashMap.get() performs a volatile read of the table array slot (array is volatile). By JMM: any put() that wrote to that slot happens-before the volatile read in get(). This ensures get() always sees the latest written value without any explicit lock. The Node.val field is also volatile -- reading it guarantees visibility of the value written by put(). The volatile read also orders all prior stores -- meaning the key.hashCode(), key.equals(), value are all visible after the volatile read.

[ADV] Q48. How does CompletableFuture handle exceptions in thenCompose chains?

If stage A completes exceptionally, thenCompose(fn) skips fn and propagates the exception downstream. fn is not called. The resulting CF completes exceptionally with the same exception. handle() and whenComplete() are called even for exceptional completion. exceptionally() recovers (returns a normal value). Common pitfall: multiple exception handlers create confusion about which stage's exception is caught. Best practice: put one handle() at the end of the chain to catch any exception from any stage, or use exceptionallyCompose() (Java 12) to recover with another async operation.

[ADV] Q49. What is the impact of using an unbounded work queue in a fixed thread pool?

Executors.newFixedThreadPool(n) uses an unbounded LinkedBlockingQueue. Under sustained overload: tasks accumulate in the queue without bound -- memory grows until OutOfMemoryError. Threads never increase beyond n (unlike CachedThreadPool). The RejectedExecutionHandler is never triggered (queue never full). Fix: use ThreadPoolExecutor directly with a bounded queue and a rejection handler. Monitor: pool.getQueue().size() to detect accumulation. Alert when queue size exceeds a threshold.

[ADV] Q50. What is a CountedCompleter in ForkJoinPool and when would you use it?

CountedCompleter is a specialisation of ForkJoinTask optimised for tree-structured tasks where a parent completes when all its children complete. It uses a pending count (setPendingCount, addToPendingCount) instead of join(). When a child completes, it decrements the parent's pending count; at 0, the parent's onCompletion() is called. Avoids the join() blocking that can cause ForkJoinPool thread starvation in large trees. Used internally by Java's Arrays.parallelSort and stream spliterator operations.

[ADV] Q51. How does `CompletableFuture.allOf()` work when one future fails?

`CompletableFuture.allOf(f1,f2,f3)` returns a `CF<Void>` that completes when ALL input CFs complete -- whether normally or exceptionally. If any CF completes exceptionally, `allOf()` itself completes exceptionally with that exception (but does NOT cancel the other CFs -- they continue running). To handle partial failures: use `.whenComplete((v,ex) -> {...})` on `allOf()`. To collect all results (including failures): use `join()` on each individual future inside the `allOf` completion handler, handling `ExecutionException` per future.

[ADV] Q52. What are the performance characteristics of `ReentrantReadWriteLock` under different read/write ratios?

`ReentrantReadWriteLock` excels when reads heavily outnumber writes. Read lock: multiple threads concurrently hold it -- no contention between readers. Write lock: exclusive -- all readers and writers block. Issues: (1) **Write starvation** in non-fair mode -- continuous readers block writers; (2) **Lock downgrading** allowed (write -> read without releasing), not upgrading; (3) read lock cannot be acquired if a write is waiting (to prevent writer starvation). For optimistic reads: `StampedLock` outperforms `ReadWriteLock` at very high read ratios since `tryOptimisticRead()` requires no lock at all.

Coding Exercises

Exercise 1: Thread-Safe Cache using ConcurrentHashMap.computeIfAbsent

```
public class AsyncCache<K, V> {
    private final ConcurrentHashMap<K, CompletableFuture<V>> cache = new ConcurrentHashMap<>();
    private final Function<K, CompletableFuture<V>> loader;
    private final Executor executor;

    public AsyncCache(Function<K, CompletableFuture<V>> loader, Executor executor) {
        this.loader = loader;
        this.executor = executor;
    }

    public CompletableFuture<V> get(K key) {
        // computeIfAbsent is atomic -- only one loader call per key
        return cache.computeIfAbsent(key, k ->
            CompletableFuture
                .supplyAsync(() -> loader.apply(k).join(), executor)
                .whenComplete((v, ex) -> {
                    if (ex != null) cache.remove(k); // don't cache failures
                })
        );
    }
}
```

Exercise 2: Bounded Thread Pool with CallerRunsPolicy and Monitoring

```
public class MonitoredThreadPool {
    private final ThreadPoolExecutor pool;

    public MonitoredThreadPool(int core, int max, int queueSize) {
        pool = new ThreadPoolExecutor(
            core, max, 60, TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(queueSize),
            r -> {
                Thread t = new Thread(r);
                t.setName("monitored-worker-" + t.getId());
                return t;
            },
            new ThreadPoolExecutor.CallerRunsPolicy()
        );
    }

    public void submit(Runnable task) { pool.execute(task); }

    public void printStats() {
        System.out.printf("Active: %d, Queued: %d, Completed: %d\n",
            pool.getActiveCount(),
            pool.getQueue().size(),
            pool.getCompletedTaskCount());
    }

    public void shutdown() throws InterruptedException {
        pool.shutdown();
        if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
            pool.shutdownNow();
        }
    }
}
```

Exercise 3: CompletableFuture Pipeline with Timeout and Fallback

```
public class ProductService {
    private final Executor ioExec = Executors.newVirtualThreadPerTaskExecutor();
```

```
public CompletableFuture<ProductDTO> getProductDetails(String id) {
    CompletableFuture<Product> productFuture =
        CompletableFuture.supplyAsync(() -> productRepo.findById(id), ioExec);
    CompletableFuture<List<Review>> reviewsFuture =
        CompletableFuture.supplyAsync(() -> reviewService.getReviews(id), ioExec);
    return productFuture
        .thenCombine(reviewsFuture, ProductDTO::new)
        .orTimeout(3, TimeUnit.SECONDS)
        .exceptionally(ex -> ProductDTO.placeholder(id));
}
}
```

Exercise 4: CountdownLatch for Parallel Initialisation

```
public class ApplicationBootstrap {
    public void initialise() throws InterruptedException {
        CountdownLatch ready = new CountdownLatch(3);
        ExecutorService exec = Executors.newFixedThreadPool(3);
        exec.submit(() -> { initDatabase(); ready.countDown(); });
        exec.submit(() -> { initCache(); ready.countDown(); });
        exec.submit(() -> { initMessaging(); ready.countDown(); });
        boolean allReady = ready.await(30, TimeUnit.SECONDS);
        exec.shutdown();
        if (!allReady) throw new RuntimeException("Bootstrap timed out");
        System.out.println("All systems ready");
    }
}
```

Debug & Fix Scenarios

Debug 1: Silent Exception in Periodic ScheduledTask

```
// BUG: exception silently cancels periodic task
ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);
scheduler.scheduleAtFixedRate(() -> {
    processItems(); // throws RuntimeException occasionally
// BUG: exception propagates out, task is silently cancelled
// No more scheduled runs after the first exception
}, 0, 10, TimeUnit.SECONDS);

// Fix: wrap in try/catch
scheduler.scheduleAtFixedRate(() -> {
    try {
        processItems();
    } catch (Exception e) {
        log.error("Periodic task failed", e); // log but don't rethrow
    }
}, 0, 10, TimeUnit.SECONDS);
```

Debug 2: CompletableFuture Exception Swallowing

```
// BUG: exception in supplyAsync is silently lost if get() is never called
CompletableFuture<String> cf = CompletableFuture.supplyAsync(() -> {
    throw new RuntimeException("Processing failed");
});

// BUG: no get(), no exceptionally(), no whenComplete() -- exception is gone
// Fix: always handle exceptions
CompletableFuture<String> safe = CompletableFuture.supplyAsync(() -> {
    throw new RuntimeException("Processing failed");
})
.exceptionally(ex -> {
    log.error("Task failed", ex);
    return "default-value"; // or rethrow
})
.whenComplete((result, ex) -> {
    if (ex != null) metrics.increment("task.failures");
});
```

Debug 3: Non-Atomic Check-Then-Act on ConcurrentHashMap

```
// BUG: two operations are NOT atomic -- race condition
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

// Thread 1 and Thread 2 both execute this simultaneously
if (!map.containsKey("counter")) { // BUG: both may see false
    map.put("counter", 1); // both may put 1, losing one increment
}

map.put("counter", map.get("counter") + 1); // BUG: read-modify-write not atomic

// Fix: use atomic operations
map.putIfAbsent("counter", 0);
map.merge("counter", 1, Integer::sum); // atomic read-modify-write
// or
map.compute("counter", (k, v) -> v == null ? 1 : v + 1); // atomic
```

Debug 4: Thread Pool with Unbounded Queue Causing OOM

```
// BUG: Executors.newFixedThreadPool uses unbounded queue -- OOM under load
ExecutorService pool = Executors.newFixedThreadPool(10);

// Under load, tasks accumulate in unbounded LinkedBlockingQueue -> OOM
// Fix: use ThreadPoolExecutor with bounded queue + rejection policy
```

```
ThreadPoolExecutor bounded = new ThreadPoolExecutor(  
    10, 10, // fixed size (no non-core threads)  
    0, TimeUnit.SECONDS,  
    new ArrayBlockingQueue<>(500), // bounded queue  
    Executors.defaultThreadFactory(),  
    (r, exec) -> { // custom rejection: log + throw  
        log.warn("Pool saturated: rejecting task");  
        throw new RejectedExecutionException("Service overloaded");  
    }  
);
```

Multiple Choice Questions

Q1. What does ThreadPoolExecutor do when the bounded queue is full and maxPoolSize is reached?

- A) Blocks the calling thread
- B) Creates a new thread
- C) Invokes the RejectedExecutionHandler [Correct]
- D) Drops the task silently

Q2. Which CompletableFuture method handles BOTH normal and exceptional completion?

- A) exceptionally()
- B) thenApply()
- C) handle() [Correct]
- D) thenAccept()

Q3. Which BlockingQueue implementation has zero capacity?

- A) ArrayBlockingQueue(0)
- B) SynchronousQueue [Correct]
- C) LinkedBlockingQueue(0)
- D) ConcurrentLinkedQueue

Q4. What is the default executor used by CompletableFuture.supplyAsync() with no executor argument?

- A) Executors.newCachedThreadPool()
- B) Executors.newFixedThreadPool(4)
- C) ForkJoinPool.commonPool() [Correct]
- D) A new single thread

Q5. How does CyclicBarrier differ from CountdownLatch?

- A) CyclicBarrier is one-shot; CountdownLatch is reusable
- B) CyclicBarrier is reusable; CountdownLatch is one-shot [Correct]
- C) CyclicBarrier counts up; CountdownLatch counts down
- D) There is no difference

Q6. What does LongAdder.sum() return under concurrent updates?

- A) An exact atomic count
- B) An approximate sum (may miss in-flight updates) [Correct]
- C) The maximum value seen
- D) It blocks until all updates complete

Quick Reference Cheat Sheet

Concept	Key Detail	Gotcha / Tip
ThreadPoolExecutor	core, max, queue, keepAlive, factory, handler	Unbounded queue = no rejection, OOM risk
shutdown()	Stop accepting; complete queued tasks	Non-blocking; use awaitTermination after
shutdownNow()	Interrupt running; drain queue to List	Tasks must respond to interrupt for clean stop
Future.get()	Blocks; wraps exception in ExecutionException	Swallows exception if never called
CompletableFuture	Non-blocking callbacks, composition, exception handling	Default executor = ForkJoinPool.commonPool()
thenCompose	Flat-map: T -> CF[U] (avoid nested CFs)	Use instead of thenApply when fn returns CF
handle(fn)	(T,ex)->U: called always for both success and failure	Vs exceptionally: only called on failure
orTimeout	CF throws TimeoutException after duration (Java 9+)	completeOnTimeout returns a value instead
ReentrantLock	Timed tryLock, interruptible, fair mode, Condition	Always unlock in finally block
StampedLock	Optimistic reads; not reentrant; not condition-based	validate(stamp) must be checked after read
LongAdder	Distributed cells; lower CAS contention than AtomicLong	sum() is NOT a consistent snapshot
ConcurrentHashMap	Per-bucket sync (Java 8); no null keys/values	size() is approximate; use mappingCount()
CopyOnWriteArrayList	Lock-free reads; O(n) writes (copy array)	Iterators show snapshot; no CME
SynchronousQueue	Zero capacity; direct thread-to-thread handoff	Used in newCachedThreadPool()
CountDownLatch	One-shot countdown; cannot be reset	Use CyclicBarrier for reusable barriers
Semaphore	N permits; release() by different thread allowed	Binary semaphore (N=1) as cross-thread signal
ForkJoinPool	Work-stealing; RecursiveTask for divide-and-conquer	I/O in commonPool blocks CPU tasks
CallerRunsPolicy	Rejected task runs on submitting thread = back-pressure	Safest rejection policy for preventing data loss